

20 Post Launch and Live Operations Plan

Documentación normativa de Cartridge & Cloud

Proyecto

Cartridge & Cloud

Versión del proyecto

0.0.26

Autor

Blas Luis Rocha González / VRM Games

Versión documental

2.0

Estado

Baseline aprobada y consolidada

Última revisión

17 de julio de 2026

Índice

20 Post Launch and Live Operations Plan	4
title: "Cartridge & Cloud — Post-Launch and Live Operations Plan" subtitle: "Plan consolidado de soporte, incidentes, parches, comunidad, métricas, mantenimiento y fin de vida" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-06" lang: es-ES document_version: "2.0" project_version: "0.0.26" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE"	4
Cartridge & Cloud — Post-Launch and Live Operations Plan	4
Resumen ejecutivo	5
Convenciones de estado	5
0. Propósito, alcance y autoridad	5
1. Genealogía histórica	6
2. Diferencia entre soporte, live operations y desarrollo futuro	6
3. Estado actual del proyecto	7
4. Principios operativos	7
5. Horario y capacidad de soporte	8
6. Canales de entrada	8
7. Email de soporte	9
8. Steam Community Hub	9
9. Foro de bugs	10
10. Foro de sugerencias	10
11. Reseñas	11
12. Redes y web	11
13. Privacidad	12
14. Datos mínimos solicitados	12
15. Logs	12
16. Saves	13
17. Consentimiento	13
18. Clasificación de incidencias	14
19. Severidades S0–S4	14
20. Prioridades	15
21. Impacto	16
22. Frecuencia	16
23. Reproducibilidad	16
24. Alcance	17
25. Triage	17
26. Duplicados	18
27. Bugs conocidos	18
28. Workarounds	19
29. Escalado	19
30. Incidentes críticos	20
31. Caídas de lanzamiento	20
32. Pérdida de save	21
33. Corrupción	21
34. Crash	22
35. Bloqueo de progreso	22
36. Economía rota	23
37. Duplicación	23
38. Localización	24
39. Audio	24
40. Rendimiento	25
41. Compatibilidad	25
42. Instalación	26
43. Actualización	26
44. Antivirus	27
45. Steam Cloud	27

46. Proton y Steam Deck	28
47. Plantilla de reporte	28
48. Evidencia mínima	29
49. Player.log	29
50. Dumps	30
51. Hardware	30
52. Sistema operativo	31
53. Idioma	31
54. Versión	32
55. Branch	32
56. Manifest ID	33
57. Save afectado	33
58. Reproducción	34
59. Registro de incidencias	34
60. SLA internos	35
61. Tiempo de primera respuesta	35
62. Tiempo de triage	36
63. Tiempo de decisión	36
64. Comunicación inicial	37
65. Actualizaciones de estado	37
66. Resolución	38
67. Cierre	38
68. Hotfix	39
69. Patch	39
70. Update	40
71. Content update	40
72. Save migration	41
73. Backward compatibility	41
74. Forward compatibility	42
75. Versionado	42
76. Branches	43
77. Release candidate	43
78. Previous stable	44
79. Rollback	44
80. Kill switch o feature flag	45
81. Build reproducible	45
82. Checksums	46
83. QA mínima de hotfix	46
84. QA completa de patch	47
85. Golden Path	47
86. Campaña de siete días	48
87. Save/load	48
88. Instalación limpia	49
89. Actualización desde versión anterior	49
90. Rollback de versión	50
91. Rollback de datos	51
92. SteamPipe	51
93. Depots	52
94. Manifest promotion	52
95. Notas de parche	53
96. Idiomas de las notas	53
97. Comunicación de riesgos	54
98. Known issues	54
99. Comunidad	55
100. Moderación	55
101. Código de conducta	56
102. Spam	56
103. Abuso	57
104. Contenido ilegal	57

105. Spoilers	58
106. Solicitudes de reembolso	58
107. Comentarios negativos	59
108. Reseñas negativas	59
109. Reseñas positivas	60
110. Respuestas públicas	60
111. Transparencia	61
112. No prometer fechas sin aprobación	61
113. Roadmap público	62
114. Roadmap interno	62
115. Sugerencias	63
116. Votaciones	63
117. Alcance futuro	64
118. DLC	64
119. Expansiones	65
120. Demo	65
121. Playtest	66
122. Achievements	66
123. Cloud	67
124. Localización futura	67
125. Nuevas plataformas	68
126. Métricas de lanzamiento	68
127. Crash-free sessions	69
128. Retención	69
129. Tiempo de juego	70
130. Abandono	70
131. Progreso	71
132. Economía	71
133. Stockouts	72
134. Softlocks	72
135. Rendimiento operativo	73
136. Soporte	73
137. Volumen de tickets	74
138. Tiempo de respuesta	74
139. Sentimiento	75
140. Reviews	75
141. Wishlists convertidas	76
142. Privacidad de métricas	76
143. Telemetría opcional	77
144. No recolectar datos sin política	77
145. Informes diarios	78
146. Informes semanales	78
147. Retrospectiva	79
148. Mantenimiento	79
149. Dependencias	80
150. Unity y paquetes	80
151. Vulnerabilidades	81
152. Actualizaciones del motor	81
153. Compatibilidad futura	82
154. Pruebas de regresión	83
155. Deuda técnica	83
156. End of life	84
157. Fin de soporte	84
158. Aviso a jugadores	84
159. Conservación de saves	85
160. Disponibilidad offline	85
161. Archivado	86
162. Work packages	86
163. Gates	87

164. Checklists	88
165. Plantillas	88
166. Riesgos	88
167. Glosario	89
168. Historial de cambios	89
Anexo A. Fuentes oficiales de Steamworks verificadas	90
Anexo B. Evidencia de implementación observada	90
Escenas de build observadas	91
Archivos técnicos hashados	91
Anexo C. Inventario de documentos consolidados	91
Anexo D. Inventario histórico y operativo preservado	91
Anexo E. Matriz de preparación postlanzamiento	91

20 Post Launch and Live Operations Plan

Metadatos normativos v2.0

document_version: 2.0
project_version: 0.0.26
last_reviewed: 2026-07-17
status: APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE
lifecycle: LIVING DOCUMENT

title: "Cartridge & Cloud — Post-Launch and Live Operations Plan" subtitle: "Plan consolidado de soporte, incidentes, parches, comunidad, métricas, mantenimiento y fin de vida" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-06" lang: es-ES document_version: "2.0" project_version: "0.0.26" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE"

Cartridge & Cloud — Post-Launch and Live Operations Plan

Proyecto: Cartridge & Cloud

Desarrollador: VRM Games / Blas Luis Rocha González

Plataforma prevista: PC / Steam

Motor: Unity 6.3 LTS 6000.3.18f1 / URP 17.3.0

Versión observada: 0.0.26

Estado: Sprints 0-15 CLOSED / PASS; Sprint 16 COMPLETED / PASS; Sprint 17 APPROVED / IMPLEMENTED / VALIDATED / CLOSED; H6 APPROVED / IMPLEMENTED / VALIDATED / CLOSED

Estado postlanzamiento: NOT OPEN

Fecha de verificación de Steamworks: 1 de julio de 2026

Aviso: este documento es un plan operativo y de gobernanza. No sustituye asesoramiento jurídico, laboral, fiscal, de protección de datos, seguridad o comunicación de crisis. Los términos y herramientas de Steamworks deben revalidarse antes de cada activación material.

Resumen ejecutivo

El proyecto todavía no está publicado y no dispone de AppID, Steamworks SDK, Community Hub, Steam Cloud, telemetría, crash reporting externo, ticketing ni dirección pública de soporte. Por tanto, este plan no describe una operación existente: define las condiciones, procedimientos, plantillas y gates que deberán estar preparados antes de aceptar jugadores públicos.

La base técnica aporta ventajas importantes: guardado local atómico, backup y recuperación; estado económico determinista; baseline de QA; versionado; builds históricas; y un plan de Steam detallado. Sin embargo, la publicación sigue bloqueada por StoreInitial, Sprint 17/H6, localización, audio, fuente, notices, clearance, soporte y autorización legal.

Área	Estado actual	
Juego público	No publicado	N
Community Hub	No existe	P
Soporte público	No existe	D
Telemetría	No integrada	N
Crash reporting	No integrado	E
Steam Cloud	No integrado	E
Save local	Atómico, backup, recovery, schema 2	B
Builds	17 PASS + 3 pendientes	N
Rollback Steam	Documentado, no probado	R
HISTORICAL EVIDENCE — infraestructura automatizada retirada:	Baseline de regresión	1

Convenciones de estado

Estado	Significado
CURRENT	Evidencia real observada en proyecto o documento vigente.
PLANNED	Trabajo aprobado como plan, aún no ejecutado.
DEFERRED	Válido, pero pospuesto por prioridad o dependencia.
NOT OPEN	Sistema o compromiso no autorizado.
BLOCKED	No puede avanzar hasta cerrar una condición.
HISTORICAL	Se conserva como genealogía, no como autoridad actual.

0. Propósito, alcance y autoridad

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **propósito, alcance y autoridad** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidatura pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.

4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

1. Genealogía histórica

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **genealogía histórica** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidatura pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

2. Diferencia entre soporte, live operations y desarrollo futuro

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **diferencia entre soporte, live operations y desarrollo futuro** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidatura pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.

6. Registrar en el ticket o release record el impacto específico de Diferencia entre soporte, live operations y desarrollo futuro y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

3. Estado actual del proyecto

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **estado actual del proyecto** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La aplicación observada es 0.0.26, Windows x64, Unity 6000.3.18f1. Sprints 0-16 y BLD-016-POST están en PASS; Sprint 17 está APPROVED / IMPLEMENTED / VALIDATED / CLOSED y H6 permanece APPROVED / IMPLEMENTED / VALIDATED / CLOSED. El registro legal cuenta 0 builds elegibles para publicación, 9 riesgos High/Critical y 7 deudas S1.

Regla operativa. No se denomina postlanzamiento a una fase que todavía no tiene producto publicado, AppID, branch pública ni autorización legal.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Estado actual del proyecto y su relación con versión, build, save y comunicación.

Evidencia mínima. Estado consolidado de producción, legal, Steam y QA con fecha y hash.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

4. Principios operativos

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **principios operativos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.

6. Registrar en el ticket o release record el impacto específico de Principios operativos y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

5. Horario y capacidad de soporte

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **horario y capacidad de soporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidatura pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Horario y capacidad de soporte y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

6. Canales de entrada

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **canales de entrada** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidatura pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.

6. Registrar en el ticket o release record el impacto específico de Canales de entrada y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

7. Email de soporte

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **email de soporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe una dirección pública documentada. Debe crearse una cuenta del dominio oficial solo después de cerrar identidad, dominio, privacidad y capacidad de atención.

Regla operativa. No se publica una dirección personal ni se usa como repositorio de saves indefinido. Se configura MFA, recuperación y retención.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Email de soporte y su relación con versión, build, save y comunicación.

Evidencia mínima. Prueba de envío/recepción, autorespuesta, acceso, backup y política de datos.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

8. Steam Community Hub

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **steam community hub** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El Hub aparecerá cuando la aplicación alcance Coming Soon. Steam permite foros específicos para bugs o idiomas y muestra como desarrollador a las cuentas con permisos adecuados.

Regla operativa. Antes de Coming Soon deben existir reglas, permisos, owner y un plan de moderación. Valve revisa contenido reportado, pero el estudio mantiene responsabilidad operativa.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Steam Community Hub y su relación con versión, build, save y comunicación.

Evidencia mínima. Captura de Hub, foros, permisos, reglas y prueba de moderación.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

9. Foro de bugs

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **foro de bugs** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Foro de bugs y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

10. Foro de sugerencias

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **foro de sugerencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Foro de sugerencias y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

11. Reseñas

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **reseñas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Las reviews son feedback sobre juego, valor, prácticas y comunidad. No todas requieren respuesta; una respuesta oficial puede amplificar una discusión.

Regla operativa. No solicitar reviews dentro del juego ni ofrecer recompensas. No refutar opiniones. Responder solo con claridad, corrección factual, solución o enlace a notas.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Reseñas y su relación con versión, build, save y comunicación.

Evidencia mínima. Registro de review, decisión de responder, wording aprobado y resultado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

12. Redes y web

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **redes y web** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidatura pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Redes y web y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

13. Privacidad

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **privacidad** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No hay telemetría, Cloud ni servicio online integrado; por ello la superficie de datos actual es principalmente local. El soporte futuro puede recibir logs y saves si el jugador los envía voluntariamente.

Regla operativa. Aplicar minimización, propósito, retención y acceso restringido. No prometer anonimato si los archivos contienen identificadores o rutas.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Privacidad y su relación con versión, build, save y comunicación.

Evidencia mínima. Inventario de datos, notice de soporte, retención y procedimiento de borrado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

14. Datos mínimos solicitados

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **datos mínimos solicitados** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidatura pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Datos mínimos solicitados y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

15. Logs

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **logs** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos,

para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El artefacto primario es `Player.log`; el proyecto también conserva logs y reportes de builds internas. No hay envío automático.

Regla operativa. El log se pide con instrucciones por plataforma y se revisa para retirar nombres de usuario, rutas o información ajena al incidente.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Logs y su relación con versión, build, save y comunicación.

Evidencia mínima. Log hashado, versión, ventana temporal y análisis asociado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

16. Saves

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **saves** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Los guardados integrados viven bajo `Application.persistentDataPath/IntegratedSaveG` usan `primary`, `.bak`, `.tmp` y `.recovery`, generación creciente y validación. El esquema actual es 2.

Regla operativa. Un save enviado no se modifica sobre la única copia. Se duplica, se trabaja en entorno aislado y se conserva cadena de custodia.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Saves y su relación con versión, build, save y comunicación.

Evidencia mínima. Save original hashado, copia de trabajo, resultado de validación y herramienta usada.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

Las pruebas usan fixtures reales anonimizados, `primary/backup` y estados límite; nunca trabajan sobre la única copia del jugador.

17. Consentimiento

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **consentimiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Consentimiento y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

18. Clasificación de incidencias

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **clasificación de incidencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidatura pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Clasificación de incidencias y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

19. Severidades S0-S4

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **severidades s0-s4** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Severidades S0–S4 y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

Severidad	Ejemplo postlanzamiento	Respuesta
S0	riesgo de seguridad, pérdida masiva de datos o build dañina	retirada/
S1	no arranca, corrupción reproducible, softlock general, exploit económico persistente	congelar
S2	feature importante rota con workaround limitado	patch pri
S3	defecto menor o localizado	backlog
S4	cosmético, sugerencia o mejora	priorizac

20. Prioridades

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **prioridades** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Prioridades y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

21. Impacto

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **impacto** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Impacto y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

22. Frecuencia

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **frecuencia** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Frecuencia y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

23. Reproducibilidad

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **reproducibilidad** después de una publicación. La regla se aplica tanto a incidencias reales como a

ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Reproducibilidad y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

24. Alcance

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **alcance** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Alcance y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

25. Triage

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **triage** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Triage y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

26. Duplicados

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **duplicados** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Duplicados y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

27. Bugs conocidos

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **bugs conocidos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Bugs conocidos y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

28. Workarounds

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **workarounds** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Workarounds y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

29. Escalado

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **escalado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Escalado y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

30. Incidentes críticos

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **incidentes críticos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Incidentes críticos y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

31. Caídas de lanzamiento

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **caídas de lanzamiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Caídas de lanzamiento y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

32. Pérdida de save

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **pérdida de save** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Es un incidente potencial S0/S1 porque afecta progreso y confianza. El repositorio puede recuperar backup, pero una pérdida más allá de primary/backup necesita procedimiento manual y comunicación prioritaria.

Regla operativa. Congelar acciones que sobrescriban archivos, pedir copia completa de la carpeta y evitar instrucciones destructivas.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Pérdida de save y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete primary/backup/recovery, logs, versión, causa y plan de restauración.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

33. Corrupción

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **corrupción** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El codec rechaza schema no soportado, generación inválida y datos inconsistentes; el repositorio intenta backup antes de declarar CorruptPrimaryNoBackup.

Regla operativa. No “arreglar” JSON a mano sin herramienta versionada, validación y backup. Toda reparación debe producir un nuevo archivo y registro.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.

5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Corrupción y su relación con versión, build, save y comunicación.

Evidencia mínima. Diagnóstico de codec, herramienta de reparación, tests y save restaurado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

34. Crash

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **crash** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe SDK externo. La función histórica Steam Error Reporting no es apropiada para el objetivo Windows x64 porque la documentación oficial la marca cercana a EOL y limitada a 32 bits.

Regla operativa. Elegir explícitamente entre soporte manual, dumps del sistema o un proveedor moderno, con revisión legal y de privacidad.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Crash y su relación con versión, build, save y comunicación.

Evidencia mínima. Crash reproducible, log/dump, símbolos, build exacta, stack y fix validado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

35. Bloqueo de progreso

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **bloqueo de progreso** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Bloqueo de progreso y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

36. Economía rota

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **economía rota** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Economía rota y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

37. Duplicación

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **duplicación** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Duplicación y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

38. Localización

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **localización** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Localización y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

39. Audio

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **audio** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Audio y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

40. Rendimiento

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **rendimiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Rendimiento y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

41. Compatibilidad

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **compatibilidad** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Compatibilidad y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

42. Instalación

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **instalación** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Instalación y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

43. Actualización

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **actualización** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Actualización y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

44. Antivirus

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **antivirus** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Antivirus y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

45. Steam Cloud

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **steam cloud** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No está implementado. El guardado actual es local y atómico. Steam Cloud puede usar Auto-Cloud o API, sincroniza antes/después de sesión y puede probarse en modo developers-only.

Regla operativa. No activar Cloud hasta probar conflictos, backup, cuota, múltiples equipos, desactivación por usuario, suspend/resume y compatibilidad de schema.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Steam Cloud y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de paths, cuotas, pruebas en dos equipos, cloud_log.txt y decisión.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

46. Proton y Steam Deck

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **proton y steam deck** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Proton y Steam Deck y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

47. Plantilla de reporte

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **plantilla de reporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa post-lanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de Plantilla de reporte y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

Título:
Versión / build / branch / manifest:
Sistema operativo y hardware:
Idioma y resolución:
Pasos exactos:
Resultado esperado:
Resultado real:
Frecuencia:
Player.log adjunto: sí/no
Save adjunto y consentimiento: sí/no
Workaround probado:
Notas:

48. Evidencia mínima

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **evidencia mínima** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de Evidencia mínima y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

49. Player.log

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **player.log** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.

6. Registrar en el ticket o release record el impacto específico de Player.log y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

50. Dumps

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **dumps** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de Player.log, builds versionadas internamente y guardados JSON con primary, .bak, .tmp y .recovery. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de Dumps y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

51. Hardware

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **hardware** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de Player.log, builds versionadas internamente y guardados JSON con primary, .bak, .tmp y .recovery. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de Hardware y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

52. Sistema operativo

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **sistema operativo** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de Sistema operativo y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

53. Idioma

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **idioma** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de Idioma y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

54. Versión

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **versión** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de Versión y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

55. Branch

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **branch** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de Branch y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

56. Manifest ID

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **manifest id** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de Manifest ID y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

57. Save afectado

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **save afectado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de Save afectado y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

58. Reproducción

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **reproducción** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de Reproducción y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

59. Registro de incidencias

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **registro de incidencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con `primary`, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de Registro de incidencias y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

Campo	Obligatorio
Incident ID	sí
Fecha/hora y zona	sí
Fuente/canal	sí
Severidad/priority	sí
Build/manifest/branch	sí
Reproducción	sí para bug
Save/log/dump hash	cuando aplique
Owner	sí
Decisión y comunicación	sí
Release que corrige	al cerrar

60. SLA internos

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **sla internos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de Player .log, builds versionadas internamente y guardados JSON con primary, .bak, .tmp y .recovery. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de SLA internos y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-0PS-012; ST-0PS-015.

Clase	Acuse objetivo	Triage objetivo	Decisión obje
S0	tan pronto como sea detectado	inmediato	retirada/contención en la misma sesión opera
S1	mismo día laborable	mismo día	siguiente ventana razonable tras reproduc
S2	1-2 días laborables	2 días	incluir o rechazar con mo
S3/S4	sin garantía pública	revisión periódica	backlog/clo

61. Tiempo de primera respuesta

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **tiempo de primera respuesta** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de Tiempo de primera respuesta y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

62. Tiempo de triage

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **tiempo de triage** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de Tiempo de triage y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

63. Tiempo de decisión

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **tiempo de decisión** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de Tiempo de decisión y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

64. Comunicación inicial

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **comunicación inicial** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de Comunicación inicial y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

65. Actualizaciones de estado

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **actualizaciones de estado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de Actualizaciones de estado y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

66. Resolución

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **resolución** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de Resolución y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

67. Cierre

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **cierre** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de Cierre y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

68. Hotfix

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **hotfix** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Un hotfix corrige un defecto urgente con alcance mínimo. No incorpora features ni refactors no esenciales.

Regla operativa. Requiere reproducción, test de regresión dirigido, save/load, instalación/actualización y rollback. Incrementa PATCH cuando es público.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de Hot fix y su relación con versión, build, save y comunicación.

Evidencia mínima. Branch hotfix, diff, pruebas, manifest, notas y autorización.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

69. Patch

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **patch** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva previous_stable, el manifest anterior y una copia de los datos necesarios. El hotfix

limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Patch y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

70. Update

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **update** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Update y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

71. Content update

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **content update** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix

limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Content update y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

72. Save migration

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **save migration** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El sistema actual solo acepta `CurrentSchemaVersion = 2`; no existe todavía una cadena de migradores históricos para una audiencia pública.

Regla operativa. Antes de cambiar schema se implementa migración versionada, idempotente, testeada desde cada versión soportada y con copia previa.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Save migration y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de schemas, migradores, fixtures reales, rollback/fallback y estadísticas de resultado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

```
load original -> validate source schema -> create immutable backup
-> migrate one version at a time -> validate invariants
-> write new temporary file -> reload and compare
-> commit atomically -> preserve recovery evidence
```

73. Backward compatibility

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **backward compatibility** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Backward compatibility y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

74. Forward compatibility

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **forward compatibility** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Forward compatibility y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

75. Versionado

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **versionado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Versionado y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

76. Branches

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **branches** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Branches y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

77. Release candidate

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **release candidate** después de una publicación. La regla se aplica tanto a incidencias reales

como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Release candidate y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

78. Previous stable

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **previous stable** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La rama conserva la última build pública demostrada, no simplemente la penúltima subida.

Regla operativa. No se sobrescribe ni se elimina hasta que la nueva versión supere la ventana de observación y compatibilidad.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Previous stable y su relación con versión, build, save y comunicación.

Evidencia mínima. Manifest, checksum, fecha, notas, saves de referencia y owner.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

79. Rollback

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **rollback** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Steam permite asignar builds anteriores, pero el rollback de binarios puede ser inseguro si el save ya migró.

Regla operativa. La decisión considera código, contenido, configuración y datos. Si no existe downgrade seguro, se prefiere hotfix forward o herramienta de recuperación.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Rollback y su relación con versión, build, save y comunicación.

Evidencia mínima. Manifest anterior, compatibilidad de save, prueba de rollback y comunicación.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

80. Kill switch o feature flag

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **kill switch o feature flag** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva previous_stable, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Kill switch o feature flag y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

81. Build reproducible

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **build reproducible** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva previous_stable, el manifest anterior y una copia de los datos necesarios. El hotfix

limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Build reproducible y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla app version + commit + Steam build + depot manifest + branch, evitando que dos artefactos distintos compartan una etiqueta ambigua.

82. Checksums

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **checksums** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva previous_stable, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Checksums y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla app version + commit + Steam build + depot manifest + branch, evitando que dos artefactos distintos compartan una etiqueta ambigua.

83. QA mínima de hotfix

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **qa mínima de hotfix** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de QA mínima de hotfix y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

84. QA completa de patch

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **qa completa de patch** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de QA completa de patch y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

85. Golden Path

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **golden path** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Golden Path y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

86. Campaña de siete días

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **campaña de siete días** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Campaña de siete días y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

87. Save/load

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **save/load** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Save/load y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

88. Instalación limpia

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **instalación limpia** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Instalación limpia y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

89. Actualización desde versión anterior

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **actualización desde versión anterior** después de una publicación. La regla se aplica tanto a

incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Actualización desde versión anterior y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

90. Rollback de versión

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **rollback de versión** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Rollback de versión y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

91. Rollback de datos

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **rollback de datos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Rollback de datos y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

92. SteamPipe

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **steampipe** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de SteamPipe y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

93. Depots

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **depots** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Depots y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

94. Manifest promotion

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **manifest promotion** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Manifest promotion y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla app version + commit + Steam build + depot manifest + branch, evitando que dos artefactos distintos compartan una etiqueta ambigua.

95. Notas de parche

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **notas de parche** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Steam ofrece un tipo específico de Patch Notes; cada build visible debe explicar magnitud y cambios relevantes.

Regla operativa. No ocultar cambios de save, known issues o riesgos. No inflar un hotfix como gran actualización.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Notas de parche y su relación con versión, build, save y comunicación.

Evidencia mínima. Notas ES/EN vinculadas a versión, manifest y ticket.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

96. Idiomas de las notas

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **idiomas de las notas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La versión 0.0.26, las builds de Sprint 17, H6 y Vertical Slice están aprobadas y validadas. SteamPipe, AppID, depots, branches y manifests públicos pertenecen a la futura fase de publicación y permanecen fuera del alcance de esta baseline.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva previous_stable, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de Idiomas de las notas y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

97. Comunicación de riesgos

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **comunicación de riesgos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Comunicación de riesgos y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

98. Known issues

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **known issues** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Known issues y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

99. Comunidad

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **comunidad** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros—DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Comunidad y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

100. Moderación

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **moderación** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Valve revisa contenido reportado, pero el proyecto debe vigilar el Hub y puede usar moderadores con permisos.

Regla operativa. Moderación por conducta y reglas publicadas. No borrar crítica legítima ni usar flags para desacuerdo.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Moderación y su relación con versión, build, save y comunicación.

Evidencia mínima. Log de moderación, regla aplicada, actor, fecha y posible apelación.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

101. Código de conducta

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **código de conducta** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Código de conducta y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

102. Spam

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **spam** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Spam y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

103. Abuso

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **abuso** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Abuso y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

104. Contenido ilegal

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **contenido ilegal** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.

4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Contenido ilegal y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

105. Spoilers

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **spoilers** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Spoilers y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

106. Solicitudes de reembolso

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **solicitudes de reembolso** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El estudio no decide reembolsos individuales de Steam. Puede consultar informes agregados y motivos en los reportes de Steamworks.

Regla operativa. Responder con información de soporte, no presionar al jugador ni prometer resultado del reembolso. Usar tendencias para corregir expectativas o fallos.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.

4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Solicitudes de reembolso y su relación con versión, build, save y comunicación.

Evidencia mínima. Informe agregado, motivos categorizados y acción de producto.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

107. Comentarios negativos

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **comentarios negativos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Comentarios negativos y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

108. Reseñas negativas

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **reseñas negativas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Una review negativa puede reflejar bugs, valor, marketing o experiencia comunitaria. Steam recomienda no responder a todas ni discutir opiniones.

Regla operativa. Responder solo cuando exista información útil, una corrección publicada o un malentendido factual importante.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.

5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Reseñas negativas y su relación con versión, build, save y comunicación.

Evidencia mínima. Decisión de respuesta y texto breve revisado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

109. Reseñas positivas

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **reseñas positivas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Reseñas positivas y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

110. Respuestas públicas

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **respuestas públicas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Respuestas públicas y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

111. Transparencia

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **transparencia** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Transparencia y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

112. No prometer fechas sin aprobación

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **no prometer fechas sin aprobación** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de No prometer fechas sin aprobación y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

113. Roadmap público

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **roadmap público** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros—DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Roadmap público y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

114. Roadmap interno

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **roadmap interno** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros—DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Roadmap interno y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

115. Sugerencias

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **sugerencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Sugerencias y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

116. Votaciones

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **votaciones** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Votaciones y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

117. Alcance futuro

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **alcance futuro** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros—DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Alcance futuro y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

118. DLC

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **dlc** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros—DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de DLC y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

119. Expansiones

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **expansiones** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Expansiones y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

120. Demo

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **demo** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Demo y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

121. Playtest

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **playtest** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Playtest y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

122. Achievements

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **achievements** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Achievements y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

123. Cloud

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **cloud** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Cloud y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

124. Localización futura

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **localización futura** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Localización futura y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

125. Nuevas plataformas

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **nuevas plataformas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Nuevas plataformas y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

126. Métricas de lanzamiento

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **métricas de lanzamiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen NOT OPEN o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de Métricas de lanzamiento y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

127. Crash-free sessions

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **crash-free sessions** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Crash-free sessions y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

128. Retención

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **retención** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Retención y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

129. Tiempo de juego

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **tiempo de juego** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Tiempo de juego y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

130. Abandono

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **abandono** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Abandono y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

131. Progreso

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **progreso** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Progreso y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

132. Economía

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **economía** después de una publicación. La regla se aplica tanto a incidencias reales como a

ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Economía y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

133. Stockouts

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **stockouts** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Stockouts y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

134. Softlocks

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **softlocks** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Softlocks y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

135. Rendimiento operativo

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **rendimiento operativo** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Rendimiento operativo y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

136. Soporte

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **soporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Soporte y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

137. Volumen de tickets

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **volumen de tickets** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Volumen de tickets y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

138. Tiempo de respuesta

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **tiempo de respuesta** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Tiempo de respuesta y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

139. Sentimiento

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **sentimiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Sentimiento y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

140. Reviews

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **reviews** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Reviews y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

141. Wishlists convertidas

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **wishlists convertidas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Wishlists convertidas y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

142. Privacidad de métricas

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **privacidad de métricas** después de una publicación. La regla se aplica tanto a incidencias reales

como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Privacidad de métricas y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

143. Telemetría opcional

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **telemetría opcional** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe instrumentación. Añadirla implica SDK, flujo de datos, proveedor, consentimiento o base aplicable, seguridad y mantenimiento.

Regla operativa. La telemetría es NOT OPEN hasta completar 26_Privacy_Data_and_Telemetry_Plan.md y una decisión formal.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Telemetría opcional y su relación con versión, build, save y comunicación.

Evidencia mínima. DPI/data map, contrato, política, opt-out, prueba y aprobación.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

144. No recolectar datos sin política

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **no recolectar datos sin política** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de No recolectar datos sin política y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

145. Informes diarios

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **informes diarios** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de Informes diarios y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: ST-OPS-003; ST-OPS-013.

146. Informes semanales

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **informes semanales** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Informes semanales y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

147. Retrospectiva

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **retrospectiva** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Retrospectiva y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

148. Mantenimiento

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **mantenimiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Mantenimiento y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

149. Dependencias

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **dependencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Dependencias y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

150. Unity y paquetes

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **unity y paquetes** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Unity y paquetes y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

151. Vulnerabilidades

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **vulnerabilidades** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Vulnerabilidades y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

152. Actualizaciones del motor

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **actualizaciones del motor** después de una publicación. La regla se aplica tanto a incidencias

reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Actualizaciones del motor y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

153. Compatibilidad futura

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **compatibilidad futura** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Compatibilidad futura y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

154. Pruebas de regresión

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **pruebas de regresión** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Pruebas de regresión y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

155. Deuda técnica

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **deuda técnica** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Deuda técnica y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

156. End of life

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **end of life** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El fin de vida no es abandono repentino; es una transición gobernada. Para un juego offline, la última build debe seguir siendo utilizable sin servicios innecesarios.

Regla operativa. Definir condiciones, periodo de aviso, soporte de saves, retirada de integraciones y archivo.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de End of life y su relación con versión, build, save y comunicación.

Evidencia mínima. Plan EOL, aviso, build final, saves, notices y archivo.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

157. Fin de soporte

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **fin de soporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Se diferencia de deslistado y de fin de venta. Puede cesar soporte activo manteniendo descarga y juego offline.

Regla operativa. No anunciarlo hasta decidir fechas, canales, excepciones de seguridad y destino de datos/tickets.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Fin de soporte y su relación con versión, build, save y comunicación.

Evidencia mínima. Aviso multilingüe, calendario, FAQ y archivo.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

158. Aviso a jugadores

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **aviso a jugadores** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Aviso a jugadores y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

159. Conservación de saves

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **conservación de saves** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Conservación de saves y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

160. Disponibilidad offline

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **disponibilidad offline** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El diseño actual es local/offline y no integra autenticación o servidores. Esa propiedad debe preservarse salvo decisión explícita.

Regla operativa. No introducir dependencia de red para iniciar, cargar, guardar o jugar el contenido base.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Disponibilidad offline y su relación con versión, build, save y comunicación.

Evidencia mínima. Prueba sin conexión desde instalación limpia y tras reinicio.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

161. Archivado

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **archivado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Archivado y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

162. Work packages

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **work packages** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de Work packages y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

ID	Paquete	Gate
OPS-WP-01	Canales y soporte público	Pre-Coming Soon
OPS-WP-02	Incident register y plantillas	H6+
OPS-WP-03	Save migration framework	Pre-release
OPS-WP-04	Steam branches y rollback drill	Release candidate
OPS-WP-05	Patch/hotfix pipeline	Release candidate
OPS-WP-06	Community rules/moderation	Coming Soon
OPS-WP-07	Patch notes ES/EN	Pre-release
OPS-WP-08	Crash evidence strategy	Pre-release
OPS-WP-09	Cloud feasibility and conflict tests	Optional feature gate
OPS-WP-10	Metrics/privacy decision	Post-privacy-plan
OPS-WP-11	Maintenance/dependency cadence	Launch
OPS-WP-12	EOL/archive plan	Before end of support

163. Gates

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **gates** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo.

1. Asignar work packages y dependencias.
2. Definir gates de Ready/Done.
3. Ensayar checklists en una release interna.
4. Versionar plantillas y glosario.
5. Actualizar historial con motivo, evidencia y rollback.
6. Registrar en el ticket o release record el impacto específico de Gates y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: ST-OPS-015.

164. Checklists

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **checklists** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo.

1. Asignar work packages y dependencias.
2. Definir gates de Ready/Done.
3. Ensayar checklists en una release interna.
4. Versionar plantillas y glosario.
5. Actualizar historial con motivo, evidencia y rollback.
6. Registrar en el ticket o release record el impacto específico de Checklists y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: ST-OPS-015.

165. Plantillas

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **plantillas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo.

1. Asignar work packages y dependencias.
2. Definir gates de Ready/Done.
3. Ensayar checklists en una release interna.
4. Versionar plantillas y glosario.
5. Actualizar historial con motivo, evidencia y rollback.
6. Registrar en el ticket o release record el impacto específico de Plantillas y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: ST-OPS-015.

166. Riesgos

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **riesgos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos

internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Los riesgos principales son save corruption, rollback incompatible, actualización defectuosa, soporte desbordado, moderación reactiva, promesas públicas, Cloud prematuro, privacidad, paquetes y pérdida de trazabilidad.

Regla operativa. Cada riesgo tiene probabilidad, impacto, trigger, owner, mitigación, contingencia y evidencia de cierre.

Procedimiento mínimo.

1. Asignar work packages y dependencias.
2. Definir gates de Ready/Done.
3. Ensayar checklists en una release interna.
4. Versionar plantillas y glosario.
5. Actualizar historial con motivo, evidencia y rollback.
6. Registrar en el ticket o release record el impacto específico de Riesgos y su relación con versión, build, save y comunicación.

Evidencia mínima. Registro de riesgos actualizado después de cada release o incidente.

Fuentes Steamworks relacionadas: ST-OPS-015.

167. Glosario

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **glosario** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo.

1. Asignar work packages y dependencias.
2. Definir gates de Ready/Done.
3. Ensayar checklists en una release interna.
4. Versionar plantillas y glosario.
5. Actualizar historial con motivo, evidencia y rollback.
6. Registrar en el ticket o release record el impacto específico de Glosario y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: ST-OPS-015.

168. Historial de cambios

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **historial de cambios** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo.

1. Asignar work packages y dependencias.
2. Definir gates de Ready/Done.
3. Ensayar checklists en una release interna.
4. Versionar plantillas y glosario.
5. Actualizar historial con motivo, evidencia y rollback.
6. Registrar en el ticket o release record el impacto específico de Historial de cambios y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: ST-OPS-015.

Anexo A. Fuentes oficiales de Steamworks verificadas

Las siguientes páginas se verificaron el **1 de julio de 2026**. Se conservan como fuentes operativas, pero deben revalidarse en Steamworks antes de activar una feature o cambiar un proceso.

ID	Fuente	URL
ST-OPS-001	Steam Community	https://partner.steamgames.com/doc/features/comm
ST-OPS-002	Community Moderation	https://partner.steamgames.com/doc/marketing/com
ST-OPS-003	User Reviews	https://partner.steamgames.com/doc/store/reviews
ST-OPS-004	Updating Game Build	https://partner.steamgames.com/doc/sdk/updating
ST-OPS-005	Branches (Betas)	https://partner.steamgames.com/doc/store/applicatio
ST-OPS-006	Builds and Manifests	https://partner.steamgames.com/doc/store/applicatio
ST-OPS-007	Uploading to Steam	https://partner.steamgames.com/doc/sdk/uploading
ST-OPS-008	Steam Cloud	https://partner.steamgames.com/doc/features/cloud
ST-OPS-009	Small Update / Patch Notes	https://partner.steamgames.com/doc/marketing/ever
ST-OPS-010	Events and Announcements	https://partner.steamgames.com/doc/marketing/ever
ST-OPS-011	Updating Your Game — Best Practices	https://partner.steamgames.com/doc/store/updates
ST-OPS-012	Steam Error Reporting	https://partner.steamgames.com/doc/features/error_r
ST-OPS-013	Developer Refund Reporting	https://partner.steamgames.com/doc/finance/refunds
ST-OPS-014	Managing Steamworks Users	https://partner.steamgames.com/doc/gettingstarted/
ST-OPS-015	Contacting Steamworks Support	https://partner.steamgames.com/doc/help/contact

Anexo B. Evidencia de implementación observada

Evidencia	Estado
Producto/versión	Cartridge & Cloud / 0.0.26
Empresa configurada	VRM Games
Resolución configurada	1024×768, resizable 0
Guardado integrado	JSON, primary + backup + temp + recovery, generación y validación
Schema	IntegratedGameStateSnapshot.CurrentSchemaVersion = 2
Steamworks	no encontrado en código/paquetes
Steam Cloud	no encontrado
Telemetría/analytics	no encontrado
Crash SDK externo	no encontrado

Evidencia	Estado
HTTP/network client de producto	no encontrado
Paquetes directos	41

Escenas de build observadas

Enabled	Escena	GUID
sí	Assets/_Project/Scenes/Bootstrap.unity	ada1e52ac6e09b9468e2109ccc7cbfc0
sí	Assets/_Project/Scenes/MainMenu.unity	b00786c6952a55d418cf16503030c766
sí	Assets/_Project/Scenes/Store.unity	45adcce906944f74bad3a0fb7d62f8e5
sí	Assets/_Project/Scenes/TestLab.unity	002b2ad62af3c614b91e32722c9452be

Archivos técnicos hashados

|---|---:|---|

Anexo C. Inventario de documentos consolidados

|---|---:|---|

Anexo D. Inventario histórico y operativo preservado

Se inventarían **241** fuentes relacionadas con release, QA, builds, soporte, incidentes, persistencia, Steam y gobernanza. No se omiten las versiones anteriores: sus hashes permiten distinguir reediciones idénticas y cambios reales.

|---|---|---|---:|---|

Anexo E. Matriz de preparación postlanzamiento

Dominio	Ready	Done
Soporte	canales, owner, privacidad, plantillas	simulacro completo y capacidad real
Incidentes	severidades, register, evidence	S0/S1 drill ejecutado
Hotfix	branch, versioning, tests	hotfix interno publicado y rollback probado
Save migration	schemas y fixtures	migración desde todas las versiones soportadas
Steam branches	AppID, depots, permisos	qa/rc/previous_stable verificadas
Comunidad	Hub, reglas, moderadores	prueba de publicación/moderación
Métricas	diccionario y privacidad	informes reproducibles sin datos excesivos
Mantenimiento	dependencias y cadence	primer ciclo de patch cerrado
EOL	criterios y archivo	aviso y build final preservados